

I BOB. NUMPY-GA KIRISH VA ASOSIY TUSHUNCHALAR

NumPy — Python'da sonli hisob-kitoblar uchun mo'ljallangan kuchli kutubxona. U yirik, ko'p o'lchamli massivlar va matritsalarini qo'llab-quvvatlaydi, shuningdek, ushbu massivlar ustida amallar bajarish uchun matematik funksiyalar to'plamini taqdim etadi. NumPy massiv obyektlari xotira jihatidan ancha samarali va Python ro'yxatlariga (lists) qaraganda yaxshiroq ishlaydi, bu esa ilmiy hisob-kitoblar, ma'lumotlar tahlili va mashinali o'rganish vazifalari uchun juda muhimdir. Qo'llanmaning NumPy-ga mo'ljallangan ushbu bobi uning asosiy xususiyatlari hamda soddadan murakkabgacha bo'lgan barcha tushunchalarni 11 ta paragrafga bo'lingan holda qamrab oladi.

1.1. Python NumPy moduli haqida umumiy tushuncha

NumPy (Numerical Python) — bu Python dasturlash tilining ilmiy hisob-kitoblar uchun asosiy kutubxonasi hisoblanadi. U 2005-yilda Travis Oliphant tomonidan yaratilgan bo'lib, bugungi kunda **Data Science**, **Machine Learning**, **Sun'iy intellekt** va muhandislik sohalarining poydevori bo'lib xizmat qiladi.

NumPy nima uchun kerak? Python'ning standart `list` (ro'yxat) obyektlari juda moslashuvchan (har xil turdagi ma'lumotlarni saqlay oladi), lekin yirik sonli ma'lumotlar bilan ishlashda juda sekinlashadi. NumPy bu muammoni **ndarray** (n-dimensional array) obykti orqali hal qiladi.

NumPy'ning asosiy afzalliklari:

- **Vektorizatsiya (Vectorization):** NumPy massivlari bilan ishlaganda `for` sikllaridan voz kechiladi. Amallar bir vaqtning o'zida butun massiv elementlari ustida bajariladi. Bu kodni soddalashtiradi va xatoliklar ehtimolini kamaytiradi.
- **Tezlik (Performance):** NumPy kodi asosan **C** va **Fortran** tillarida yozilgan. Bu esa standart Python ro'yxatlariga qaraganda hisoblash tezligini **10 dan 100 martagacha** oshiradi.
- **Xotira samaradorligi:** Ro'yxatlardan farqli o'laroq, NumPy massivlari xotirada ketma-ket (contiguous) bloklarda saqlanadi. Bu protsessor keshidan (CPU cache) samarali foydalanish imkonini beradi.

Massiv o‘lchamlari (Dimensions) NumPy’da massivlar o‘lchamiga qarab quyidagicha klassifikatsiya qilinadi:

- 1. **0-D Massivlar (Skalyar):** Oddiy sonlar (masalan: 42).
- 2. **1-D Massivlar (Vektor):** Elementlar ketma-ketligi (ro‘yxat kabi).
- 3. **2-D Massivlar (Matritsa):** Satr va ustunlardan iborat jadval ko‘rinishidagi ma’lumotlar.
- 4. **3-D va undan yuqori (Tensorlar):** Bir nechta matritsalarining jamlanmasi (masalan, rangli tasvirlar RGB kanallari bilan).

NumPy’ning asosiy funksional imkoniyatlari:

Imkoniyat	Tavsif
Matematik funksiyalar	Trigonometriya, logarifmlar va eksponentalar bilan ishlash uchun tayyor uskunalar.
Chiziqli algebra	Matritsalarini ko‘paytirish, determinant hisoblash, teskari matritsani topish va o‘zli qiymatlarni (eigenvalues) aniqlash.
Statistik tahlil	o‘rtacha qiymat (mean), medianana, standart og‘ish (std) va dispersiyani tezkor hisoblash.
Tasodifiy sonlar	Turli xil taqsimotlar (normal, binomial, uniform) bo‘yicha tasodifiy sonlar generatsiyasi.
Broadcasting	Turli o‘lchamdagi massivlar ustida arifmetik amallarni bajarish mexanizmi.

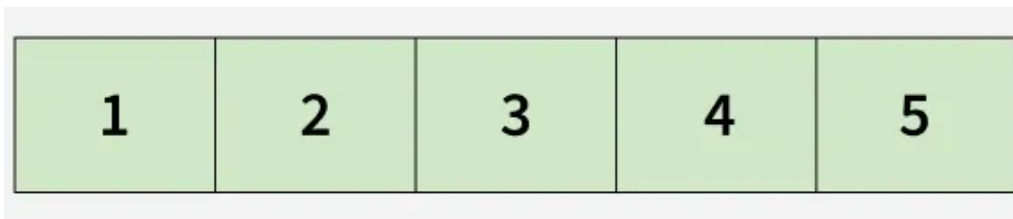
NumPy nafaqat ma’lumotlarni saqlash vositasi, balki murakkab matematik algoritmlarni kompyuter resurslaridan maksimal darajada foydalangan holda ijro etuvchi kuchli dvigateldir. Uning yordamisiz zamonaviy **Big Data** (Katta ma’lumotlar) tahlilini tasavvur qilib bo‘lmaydi.

1.2. Massiv turlari (Types of Array)

Massivlarning turli xillari quyidagilardan iborat:

1. Bir o‘lchamli massiv

Bir o‘lchamli massiv — bu chiziqli massivning bir turi.



1.1-rasm. Bir o'lchamli (1D) massiv.

```
import numpy as np

a = [1, 2, 3, 4]

arr = np.array(a)

print("Pythondagi ro'yxat : ", a)

print("Pythondagi Numpy massivi :", arr)
```

```
Pythondagi ro'yxat : [1, 2, 3, 4]
Pythondagi Numpy massivi : [1 2 3 4]
```

Ro'yxat va massiv uchun ma'lumot turini tekshirish:

```
a = [1, 2, 3, 4]
arr = np.array(a)

# Ro'yxat turini tekshirish
print(type(a))

# Numpy massiv turini tekshirish
print(type(arr))
```

```
<class 'list'>
<class 'numpy.ndarray'>
```

2. Ko'p o'lchamli massiv

Ko'p o'lchamli massiv — bu ma'lumotlarni satrlar va ustunlar kabi birdan ortiq o'lchamda saqlashi mumkin bo'lgan massivdir. Oddiy tilda aytganda, bu massivlar ichidagi massivdir. Masalan, 2D massiv satr va ustunlardan iborat jadvalga o'xshaydi, unda har bir elementga ikkita indeks orqali murojaat qilinadi: biri satr uchun, ikkinchisi ustun uchun. 3D massivlar kabi yuqori o'lchamlar qo'shimcha qatlamlar qo'shishni nazarda tutadi.

1	2	3	4	5
6	7	8	9	10

1.2-rasm. Ikki o'lchamli (2D) massiv.

```
list_1 = [1, 2, 3, 4]
list_2 = [5, 6, 7, 8]
list_3 = [9, 10, 11, 12]

sample_array = np.array([list_1,
                          list_2,
                          list_3])
print("Pythonda Numpy ko'p o'lchamli massivi\n", sample_array)
```

```
Pythonda Numpy ko'p o'lchamli massivi
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

Eslatma: ko'p o'lchamli massivlar uchun `numpy.array()` ichida `[ ]` operatorlaridan foydalaning.

Numpy massivining parametrlari

1. Axis (O'q): Massiv o'qi massivga indekslash tartibini tavsiflaydi. Axis 0 = bir o'lchamli Axis 1 = ikki o'lchamli Axis 2 = uch o'lchamli

1.3. NumPy massivining parametrlari (Parameters of a NumPy Array)

1. **Axis:** O'q massivga indekslash tartibini tavsiflaydi.

Axis 0 = bir o'lchamli

Axis 1 = ikki o'lchamli

Axis 2 = uch o'lchamli

2. Shape: Har bir o'q bo'yicha elementlar soni va u kortej (tuple) sifatida qaytariladi.

Misol:

```
sample_array = np.array([[0, 4, 2],
                          [3, 4, 5],
                          [23, 4, 5],
                          [2, 34, 5],
                          [5, 6, 7]])

print("massivning o'lchami (shape) :",
      sample_array.shape)
```

massivning o'lchami (shape) : (5, 3)

3. Rank: Massivning rangi (rank) shunchaki undagi o'qlar yoki o'lchamlar sonidir.

Bir o'lchamli massiv 1-rangga ega.

Ikki o'lchamli massiv 2-rangga ega.

4. Ma'lumot turi obyektlari (dtype): Ma'lumot turi obyektlari (dtype) `numpy.dtype` sinfining (klassining) bir misolidir. U massiv elementiga mos keladigan xotiraning belgilangan o'lchamdagi blokidagi baytlar qanday talqin qilinishi kerakligini tavsiflaydi.

NumPy-da **dtype** massivda saqlanadigan ma'lumotlar turini va har bir qiymat qancha xotira ishlatishini belgilaydi. U xom xotira baytlari qanday talqin qilinishini nazorat qiladi, bu esa NumPy operatsiyalarini tez va samarali qiladi. **dtype**-ni tushunish samaradorlik va xotiradan foydalanishni boshqarishga yordam beradi.

Ushbu misol NumPy massivining ma'lumot turini qanday aniqlashni ko'rsatadi.

```
arr = np.array([1, 2, 3])
print(arr.dtype)
```

int64

Izoh: `arr.dtype` atributi massiv elementlarining ma'lumot turini qaytaradi, bu holatda u **int** (butun son), chunki barcha qiymatlar butun sonlardir.

dtype obyektini yaratish

dtype obyekti `numpy.dtype` sinfining nusxasidir. Siz uni `np.dtype()` funksiyasidan foydalanib yaratishingiz mumkin.

```
print(np.dtype(np.int16))
```

```
int16
```

Tushuntirish:

`np.int16` 16 bitli butun sonni anglatadi `np.dtype()` uni dtype obyektiga aylantiradi

Bayt tartibi va hajmini tushunish

NumPy ma'lumotlarni xotirada xom baytlar sifatida saqlaydi, bunda bayt tartibi baytlarning qanday joylashishini nazorat qiladi va hajmi har bir qiymat uchun xotira sarfini belgilaydi. Bu turli tizimlarda ma'lumotlarning to'g'ri va samarali ishlanishini ta'minlaydi.

```
dt = np.dtype('>i4')
print("Bayt tartibi:", dt.byteorder)
print("Hajmi:", dt.itemsize)
print("Ma'lumot turi:", dt.name)
```

```
Bayt tartibi: >
```

```
Hajmi: 4
```

```
Ma'lumot turi: int32
```

Izoh:

> **big-endian** bayt tartibini bildiradi, ya'ni eng muhim bayt xotirada birinchi bo'lib saqlanadi. `i4` **4 baytli** butun son ma'lumot turini anglatadi. `itemsize` bitta qiymatni saqlash uchun qancha bayt ishlatilishini ko'rsatadi, bu **int32** uchun 4 baytni tashkil qiladi. Umuman olganda, bu massiv 32 bitli butun sonlarni **big-endian** bayt tartibidan foydalangan holda saqlashini va har bir qiymat xotirada 4 bayt joy egallashini anglatadi.

NumPy-dagi umumiy tur spetsifikatorlari (Type Specifiers)

Ba'zi ko'p ishlatiladigan tur kodlari quyidagilardir:

`i1, i2, i4, i8` -> ishorali butun sonlar (Ham manfiy, ham musbat sonlarni saqlashi mumkin, masalan: -5, 0, 10)

`u1, u2, u4, u8` -> ishorasiz butun sonlar (Faqat manfiy bo'lmagan sonlarni saqlashi mumkin, masalan: 0, 5, 100)

`f4, f8` -> suzuvchi nuqtali (o'nli kasr) sonlar (o'nli kasr qiymatlarni saqlaydi, masalan: 3.14, -0.75)

`c8, c16` -> kompleks sonlar (Haqiqiy va mavhum qismlardan iborat sonlarni saqlaydi, masalan: 2+3j)

`a, U` -> satrlar (Matnli qiymatlarni saqlaydi, masalan: "NumPy")

type va dtype o'rtasidagi farq

NumPy-da **type** va **dtype** turli maqsadlarga xizmat qiladi va ko'pincha yangi boshlovchilarni chalkashtirib yuboradi. **type** obyektning o'zi nima ekanligini tavsiflaydi (masalan, NumPy massivi), **dtype** esa massiv ichida saqlangan ma'lumotlar turini tavsiflaydi.

```
a = np.array([1])
print("type:", type(a))
print("dtype:", a.dtype)
```

```
type: <class 'numpy.ndarray'>
dtype: int64
```

Izoh:

`type(a)` **a** obykti NumPy massiv obykti ekanligini ko'rsatadi. `a.dtype` massiv 64 bitli butun sonlarni saqlashini ko'rsatadi. Bitta obyekt faqat bitta turga (type) ega bo'lishi mumkin, ammo uning **dtype**-i saqlangan ma'lumotlarni nazorat qiladi.

dtype yordamida tuzilmali massivlar (Structured Arrays)

Ba'zan bitta massivda bir nechta turdagi ma'lumotlarni birga saqlash zarurati tug'iladi, masalan, ism va sonli qiymatlar. NumPy buni *tuzilmali massivlar* (structured arrays) yordamida amalga oshiradi, bunda har bir element nomlangan maydonlarga ega kichik yozuv kabi ishlaydi.

Tuzilmali dtype-ni aniqlash

Ushbu misolda biz ikkita maydondan iborat maxsus ma'lumot turini aniqlaymiz: matnli maydon va sonli maydon.

```
dt = np.dtype([('name', np.str_, 16), ('scores', np.float64, (2,))])
print(dt['name'])
print(dt['scores'])
```

```
<U16  
( '<f8', (2,))
```

Izoh:

name — 16 tagacha belgini saqlashi mumkin boʻlgan satrli (string) maydon.

scores — ikkita haqiqiy sonni kichik massiv koʻrinishida saqlaydi.

Har bir maydon uning nomi va maʼlumot turi yordamida aniqlanadi.

Tuzilmali massiv yaratish

Bu yerda biz har bir qatori yuqorida aniqlangan tuzilmali **dtype**-ga amal qiladigan massiv yaratamiz.

```
dt = np.dtype([ ('name', np.str_, 16), ('scores', np.float64, (2,)) ])
data = np.array([ ('Emma', (8.5, 7.0)), ('Lucas', (6.0, 7.5)) ], dtype=dt)

print(data[1])
print("Ballar:", data[1]['scores'])
print("Ismlar:", data['name'])

('Lucas', [6.0, 7.5])
Ballar: [6.  7.5]
Ismlar: ['Emma' 'Lucas']
```

Izoh:

Maʼlumotlarning har bir elementi ham ismni (**name**), ham ballarni (**scores**) oʻz ichiga oladi. `data[1]` ikkinchi yozuvga murojaat qiladi. Maydonlarga ularning nomlari orqali murojaat qilinadi, masalan: `data['name']`. Tuzilmali massivlar xuddi nomlangan ustunlarga ega jadval qatorlari kabi ishlaydi.

1.4. NumPy massivini yaratishning turli usullari (Different Ways of Creating NumPy Array)

1. `numpy.array()`: Numpy-dagi massiv obyekti **ndarray** deb ataladi. Biz ushbu funksiya yordamida **ndarray** yaratishimiz mumkin.

Sintaksis: `numpy.array(parameter)`

Misol:

```
arr = np.array([3, 4, 5, 5])  
  
print("Massiv :", arr)
```

Massiv : [3 4 5 5]

2. **numpy.fromiter()** : **fromiter()** funksiyasi takrorlanuvchi (iterable) obyektidan foydalangan holda yangi bir o'lchamli massiv hosil qiladi.

Sintaksis: `numpy.array(iterable, dtype, count=-1)`

Misol:

```
# Takrorlanuvchi obyekt sifatida o'zbekcha so'zdan foydalanamiz  
matn = "Ma'lumotlar"  
  
# Har bir belgini alohida element sifatida massivga o'tkazamiz  
arr = np.fromiter(matn, dtype = 'U1')  
  
print("fromiter() yordamida yaratilgan massiv :", arr)
```

fromiter() yordamida yaratilgan massiv : ['M' 'a' '' 'l' 'u' 'm' 'o'
't' 'l' 'a' 'r']

Izoh:

matn : Massivga aylantiriladigan satrli obyekt.
dtype = 'U1' : Har bir element 1 ta Unicode belgidan iborat bo'lishini belgilaydi. Bu usul yirik hajmli ma'lumotlar oqimi bilan ishlashda xotirani tejash uchun juda samaralidir.

3. **numpy.arange()** : Bu NumPy'ning o'rnatilgan funksiyasi bo'lib, berilgan oraliqda bir xil qadam bilan joylashgan qiymatlarni qaytaradi.

Sintaksis:

`numpy.arange( boshlanish, tugash, qadam, dtype=None )`

Misol:

```
# 1 dan 25 gacha bo'lgan sonlarni 3 qadam bilan yaratamiz  
# Eslatma: 'tugash' qiymati (25) massivga kirmaydi  
arr = np.arange(1, 25, 3, dtype = np.float32)  
  
print("arange() yordamida yaratilgan massiv:\n", arr)
```

arange() yordamida yaratilgan massiv:
[1. 4. 7. 10. 13. 16. 19. 22.]

Izoh:

1 (start): Ketma-ketlikning boshlanishi.

25 (stop): Ketma-ketlikning oxiri (bu qiymat natijaga kirmaydi).

3 (step): Qiymatlar orasidagi masofa (qadam).

dtype: Massiv elementlarining turi (bu misolda o'nli kasr sonlar).

4. **numpy.linspace()**: Ushbu funksiya belgilangan ikki chegara oralig'ida bir xil masofada joylashgan sonlarni qaytaradi. **arange()** dan farqi shundaki, bu yerda qadam hajmi emas, balki nechta element kerakligi ko'rsatiladi.

Sintaksis:

```
numpy.linspace(boshlanish, tugash, soni=50, endpoint=True, retstep=False)
```

Misol:

```
# 5 dan 15 gacha bo'lgan oraliqda aniq 4 ta son hosil qilamiz
# Bunda oraliq avtomatik ravishda teng qismlarga bo'linadi
arr = np.linspace(5, 15, 4, dtype = np.float64)

print("linspace() yordamida yaratilgan massiv:\n", arr)
```

```
linspace() yordamida yaratilgan massiv:
[ 5.          8.33333333 11.66666667 15.          ]
```

Izoh:

boshlanish (5): Ketma-ketlikning boshlang'ich qiymati.

tugash (15): Ketma-ketlikning yakuniy qiymati.

soni (4): Hosil qilinishi kerak bo'lgan elementlar soni.

endpoint=True: Agar **True** bo'lsa, "tugash" qiymati massivga kiritiladi (standart holatda **True**).

5. **numpy.empty()**: Ushbu funksiya berilgan shakl (o'lcham) va turga ega bo'lgan, lekin qiymatlari boshlang'ich holatga keltirilmagan yangi massiv yaratadi. Bu usul juda tez ishlaydi, chunki u xotiradagi mavjud "axlat" qiymatlarni tozalab o'tirmaydi.

Sintaksis: `numpy.empty(shakl, dtype=float, order='C')`

Misol:

```
# 2 qator va 4 ustunli massiv yaratamiz
# order='F' (Fortran tartibi) - ma'lumotlarni ustunlar bo'yicha joylash
arr = np.empty([2, 4], dtype = np.int16, order = 'F')

print("empty() yordamida yaratilgan massiv:\n", arr)

empty() yordamida yaratilgan massiv:
[[ 69 109 109  97]
 [  0   0   0   0]]
```

Eslatma: Natijadagi sonlar sizda boshqacha chiqishi mumkin, chunki ular xotiradagi tasodifiy qoldiq qiymatlardir.

Izoh:

shakl ([2, 4]): 2 ta qator va 4 ta ustun.

order = 'F': Ma'lumotlarning xotirada joylashish tartibi. 'C' - qatorlar bo'yicha (C-style), 'F' - ustunlar bo'yicha (Fortran-style). Bu parametr katta massivlar bilan ishlashda hisoblash tezligiga ta'sir qilishi mumkin.

6. **numpy.ones()** : Ushbu funksiya berilgan shakl (o'lcham) va turga ega bo'lgan, barcha elementlari bir (1) soni bilan to'ldirilgan yangi massiv yaratish uchun ishlatiladi.

Sintaksis: `numpy.ones(shakl, dtype=None, order='C')`

Misol:

```
# 4 ta qator va 3 ta ustundan iborat massiv yaratamiz
# Barcha elementlar 1 ga teng bo'ladi
arr = np.ones([4, 3], dtype = np.int32, order = 'f')

print("ones() yordamida yaratilgan massiv:\n", arr)

ones() yordamida yaratilgan massiv:
[[1 1 1]
 [1 1 1]
 [1 1 1]
 [1 1 1]]
```

Izoh:

shakl ([4, 3]): Massivning o'lchami (4 qator, 3 ustun).

dtype = np.int32: Elementlar butun son turida bo'lishini belgilaydi.

order = 'f': Ma'lumotlarni xotirada Fortran tartibida (ustunlar bo'yicha) joylashtiradi.

ones() funksiyasi ko'pincha matematik hisoblashlarda massivlarni boshlang'ich qiymat bilan to'ldirish uchun juda qulaydir.

7. **numpy.zeros()**: Ushbu funksiya berilgan shakl (o'lcham) va turga ega bo'lgan, barcha elementlari nol (0) bilan to'ldirilgan yangi massiv yaratish uchun ishlatiladi.

Sintaksis: `numpy.zeros(shakl, dtype=None, order='C')`

Misol:

```
# 4 ta qator va 3 ta ustunli massiv yaratamiz
# Barcha qiymatlar 0 ga teng bo'ladi
massiv = np.zeros([4, 3], dtype = np.int32, order = 'f')
```

```
print("zeros() yordamida yaratilgan massiv:\n", massiv)
```

```
zeros() yordamida yaratilgan massiv:
[[0 0 0]
 [0 0 0]
 [0 0 0]
 [0 0 0]]
```

Izoh:

shakl ([4, 3]): Massiv 4 ta qator va 3 ta ustundan iborat.

dtype = np.int32: Ma'lumotlar turi sifatida butun son tanlangan.

order = 'f': Xotirada ustunlar bo'yicha tartiblash.



Nazorat savollari

1-daraja: Asosiy tushunchalar

1. **NumPy** so'zining ma'nosi nima va bu kutubxona asosan nima uchun ishlatiladi?
2. **ndarray** nima va u Python-ning standart ro'yxatlaridan (lists) qanday farq qiladi?

3. NumPy massivlari qanday o'lchamlarda bo'lishi mumkin? Har biriga bittadan misol keltiring.
4. Massivning **o'qi (axis)** nima va u 0, 1, 2 qiymatlarida qanday o'lchamlarni ifodalaydi?

2-daraja: Massiv parametrlari va turlari

5. Massivning **shakli (shape)** va **rangi (rank)** deganda nima tushuniladi?
6. **dtype** nima va u xotiradagi ma'lumotlarni boshqarishda qanday rol o'ynaydi?
7. `type()` va `dtype` buyruqlari o'rtasidagi asosiy farq nimadan iborat?
8. **Tuzilmali massivlar** (Structured Arrays) nima va ular qanday holatlarda qo'llaniladi?

3-daraja: Funksiyalar bilan ishlash

9. `np.arange()` funksiyasi yordamida massiv yaratilganda, **stop** (tugash) qiymati massivga kiritiladimi?
10. `np.linspace()` funksiyasi `np.arange()` dan qaysi jihati bilan farq qiladi?
11. `np.empty()` funksiyasi yaratgan massiv nima uchun "axlat" qiymatlardan iborat bo'ladi?
12. `order='C'` va `order='F'` parametrlari ma'lumotlarni xotirada qanday tartibda joylashtiradi?